

POLITÉCNICO DO PORTO

INSTITUTO SUPERIOR DE ENGENHARIA DO PORTO

LICENCIATURA EM

ENGENHARIA DE TELECOMUNICAÇÕES E INFORMÁTICA

## **Análise de Requisitos de um Sistema de Gestão de Camiões, Camionistas, Cargas e Entregas**

FUNDAMENTOS DE DESENVOLVIMENTO DE SOFTWARE

GRUPO 4 1DB:

AFONSO TEIXEIRA 1250717

BRUNO SARAIVA 1250795

GONÇALO VILAS BOAS 1250970

MARCO CARDOSO 1251212

Professor Paulo Baltarejo Sousa



Departamento de Engenharia Eletrotécnica

Instituto Superior de Engenharia do Porto

26 de abril a 17 de maio de 2026

# Índice

|                                                |    |
|------------------------------------------------|----|
| 1. Resumo .....                                | 3  |
| 2. Introdução.....                             | 3  |
| 3. Implementação .....                         | 4  |
| 3.1 DTO's .....                                | 4  |
| 3.2 Mapper .....                               | 5  |
| 3.3 Repository .....                           | 5  |
| 3.4 GRASP .....                                | 6  |
| 3.4.1 Creator .....                            | 6  |
| 3.4.2 Controller .....                         | 7  |
| 3.4.3 Singleton .....                          | 8  |
| 3.5 MCV (Model-View-Controller) .....          | 9  |
| 3.5.1 Model.....                               | 9  |
| 3.5.2 View .....                               | 10 |
| 3.5.3 Controller .....                         | 10 |
| 4. Testes .....                                | 11 |
| 4.1 Tabela de taxa de sucesso dos testes ..... | 14 |
| 5. Conclusão.....                              | 14 |

## **1. Resumo**

Este relatório serve de suporte escrito para o design de um projeto de desenvolvimento de software orientado a objetos em C++. O objetivo deste relatório é clarificar partes importantes do código e explicar as decisões tomadas, enfatizando a importância de compreender o software e a sua implementação.

## **2. Introdução**

O presente projeto consiste no desenvolvimento de uma aplicação de gestão de frota logística, tendo como atores principais o Gestor e o Camionista. Cada ator tem acesso a um conjunto de funcionalidades específicas, definidas de acordo com o seu papel no sistema.

O Gestor é responsável por registar e gerir as entidades centrais do sistema: camiões, camionistas e cargas. Cada camião é identificado pela sua matrícula e capacidade máxima, cada camionista pelo seu nome, e cada carga pelo seu peso e destino. Todas as entidades são inicializadas automaticamente com o estado "Disponível" aquando do seu registo. O Gestor pode ainda associar um camionista a um camião, desde que ambos se encontrem disponíveis, e remover qualquer uma destas entidades do sistema, respeitando as regras de negócio que garantem a integridade e segurança dos dados. É também possível ao Gestor consultar o histórico de entregas realizadas, visualizando as rotas concluídas com os respetivos detalhes.

O Camionista pode gerir as cargas atribuídas ao seu camião, selecionando cargas disponíveis desde que o peso total não exceda a capacidade restante do veículo. Após selecionar as cargas, o camionista pode iniciar a entrega, momento em que o sistema calcula a distância total da rota seguindo a ordem de seleção das cargas (FIFO) e apresenta o itinerário. Após a confirmação da entrega, o sistema marca todas as cargas transportadas como "Entregue", repõe o camionista e o camião no estado "Disponível" e regista a rota como concluída.

### 3. Implementação

#### 3.1 DTO's

É utilizado no GestorController para receber e trabalhar com os dados vindos do GestorService sem nunca aceder diretamente aos objetos de domínio. O Controller não conhece a classe *Camionista* nem a classe *Camiao*, apenas os respetivos DTO's que são structs simples com campos diretos. Isto é importante porque o Controller tem a responsabilidade de apresentar dados e recolher inputs do utilizador, não de manipular lógica de negócio. No primeiro excerto, o Controller pede a lista de camionistas disponíveis e passa-a diretamente à View. No segundo, quando o utilizador seleciona um camião para remover, o Controller acede ao campo *matricula* diretamente do DTO para identificar o elemento e devolve apenas esse valor ao Service.

```
117 // UC 9.4 - Atribuir Camionista a Camiao
118 else if(opcao == 4){
119     std::vector<CamionistaDTO> camionistasDisp = service->getCamionistasDisponiveis();
120
121     if(camionistasDisp.empty()){
122         menu.mostrarErro("Nao existem camionistas disponiveis.");
123         continue;
124     }
125
126     menu.mostrarCamionistasDisponiveis(camionistasDisp);
```

Figura 1 - Excerto de código do GestorController.cpp

```
176 // UC 9.5 - Remover Camiao
177 else if(opcao == 5){
178     std::vector<CamiaoDTO> camioes = service->getTodosCamioes();
```

```
...
195     std::string matricula = camioes[indice-1].matricula;
196
197     bool confirmacao = menu.pedirConfirmacao();
198     if(confirmacao){
199         try{
200             service->removerCamiao(matricula);
201             std::vector<CamiaoDTO> camioesAtualizados = service->getTodosCamioes();
202             menu.mostrarSucessoRemoverCamiao(camioesAtualizados);
203         }
204         catch(std::invalid_argument &e){
205             menu.mostrarErro(e.what());
206         }
207         break;
208     }
```

Figura 2 - Excerto de código do GestorController.cpp

### 3.2 Mapper

É utilizado no `GestorService.cpp` para converter objetos de domínio em DTO's no momento em que os dados precisam de sair da camada de negócio em direção ao Controller. O `GestorService` não sabe como construir um DTO, atribui essa responsabilidade ao Mapper correspondente. Sem os Mappers, a lógica de transformação estaria dispersa pelo código. No excerto a baixo, o Service percorre todos os camionistas, filtra os que não têm caminhão atribuído e, para cada um, chama o `CamionistaMapper::toDTO()` que extrai apenas os campos necessários para a View, sem expor ponteiros internos nem lógica de domínio.

```
124  std::vector<CamionistaDTO> GestorService::getCamionistasDisponiveis(){
125      // Obtem referencia ao vetor de todos os camionistas
126      std::vector<Camionista>& todos = camionistaContainer->getTodos();
127      std::vector<CamionistaDTO> dtos;
128
129      // Filtra: so adiciona os que ainda nao tem camiao
130      for(int i = 0; i < todos.size(); i++){
131          if(todos[i].getCamião() == nullptr){
132              dtos.push_back(CamionistaMapper::toDTO(todos[i]));
133          }
134      }
135      return dtos;
136  }
```

Figura 3 - Excerto de código do `GestorService.cpp`

### 3.3 Repository

É utilizado para isolar toda a lógica de acesso a ficheiros numa única classe, de forma a que o resto da aplicação nunca precise de saber que existem ficheiros `.txt`. O `GestorService` e o `GestorController` trabalham apenas com objetos em memória, ou seja, não têm qualquer conhecimento como esses dados foram carrados ou como são guardados. Toda essa responsabilidade pertence ao `GeneralRepository`. No `main.cpp`, o repositório é instanciado logo após o Singleton. O método `carregar()` lê os ficheiros e povoa os containers da Empresa. A partir desse momento, toda a aplicação corre sobre dados em memória. No final, `guardar()` escreve o estado atual de volta nos ficheiros. Esta separação permite, por exemplo, mudar de ficheiros `.txt` para uma base de dados sem alterar nenhuma outra classe.

```
23  GeneralRepository repository;
24  repository.carregar();
```

...

```
29  MenuPrincipalController controller(&camionistaService, &gestorService);
30  controller.mostrarMenu();
31
32  repository.guardar();
```

Figura 4 - Excerto de código do `main.cpp`

### 3.4 GRASP

#### 3.4.1 Creator

O padrão Creator é visível em dois sítios principais. No GestorService cria Camiao, Camionista e Carga, porque é quem recebe os dados de inicialização vindos do utilizador (via Controller) e quem sabe o que fazer com os dados de seguida, que será guardá-los no respetivo container.

```
26 void GestorService::registrarCamiao(std::string matricula, float capacidade){
27     Camiao camiao(matricula, capacidade); // construtor do Camiao valida tudo
28     camiaoContainer->guardar(camiao);
29 }
30
31 void GestorService::registrarCamionista(std::string nomeCamionista){
32     if(nomeCamionista.empty()){
33         throw std::invalid_argument("Nome nao pode estar vazio.");
34     }
35     if(camionistaContainer->validarNome(nomeCamionista) != nullptr){//ja existe
36         throw std::invalid_argument("Camionista ja existente");
37     }
38     Camionista camionista(nomeCamionista);
39     camionistaContainer->guardar(camionista);
40 }
41
42 void GestorService::registrarCarga(float peso, std::string nomeDestino){
43     Localidade* destino = localidadeContainer->procurar(nomeDestino);
44     if(destino == nullptr){
45         throw std::invalid_argument("Localidade destino nao encontrada.");
46     }
47     Carga carga(peso, destino);
48     cargaContainer->guardar(carga);
49 }
```

Figura 5 – Excerto de código do GestorService.cpp

No GeneralRepository cria os objetos durante os carregamentos dos ficheiros, porque é quem lê os dados em bruto e tem toda a informação necessária para construir os objetos.

```
133     Camionista camionista(nome);
134     camionista.setEstado(estado);
135
136     if(!matriculaCamiao.empty()){
137         Camiao* camiao = camiaoContainer->procurar(matriculaCamiao);
138         if(camiao != nullptr){
139             camionista.setCamiao(camiao);
140         }
141     }
142
143     camionistaContainer->guardar(camionista);
```

Figura 6 - Excerto de código do GeneralRepository

Em ambos os casos, o Creator tem acesso imediato aos dados necessários para a criação e entrega do objeto criado a quem o vai armazenar, que é precisamente o que o padrão prescreve.

### 3.4.2 Controller

Existem três controllers: MenuPrincipalController recebe a escolha inicial (gestor ou camionista) e delega para o controller correto; GestorController trata todos os eventos do gestor (registrar caminhão, registrar camionista, registrar carga, atribuir camionista a caminhão, etc.); CamionistaController trata todos os eventos do camionista (adicionar carga, remover carga, iniciar entrega, etc.).

```
10 void MenuPrincipalController::mostrarMenu() {
11     while(true) {
12         int opcao = menu.mostrar();
13
14         if(opcao == 1) {
15             GestorController gestorController(gestorService);
16             gestorController.mostrarMenu();
17         }
18         else if(opcao == 2) {
19             CamionistaController camionistaController(camionistaService);
20             camionistaController.mostrarMenu();
21         }
22         else if(opcao == 0) {
23             return;
24         }
25     }
26 }
```

Figura 7 - Excerto do código de MenuPrincipalController.cpp

O GestorController ilustra bem o papel do controller: recebe o input (do menu), valida o que pode ser validado a nível de fluxo, chama o serviço e devolve feedback à view, nunca acedendo diretamente ao modelo.

```
118 else if(opcao == 4){
119     std::vector<CamionistaDTO> camionistasDisp = service->getCamionistasDisponiveis();
120
121     if(camionistasDisp.empty()){
122         menu.mostrarErro("Nao existem camionistas disponiveis.");
123         continue;
124     }
125
126     menu.mostrarCamionistasDisponiveis(camionistasDisp);
127
128     std::string nomeCamionista;
129     bool voltar = false;
130     while(true){
131         nomeCamionista = menu.pedirNomeCamionista();
132         if(nomeCamionista == "v" || nomeCamionista == "V"){ voltar = true; break; }
133         try{
134             service->verificarCamionista(nomeCamionista);
135             break;
136         }
137         catch(std::invalid_argument &e){
138             menu.mostrarErro(e.what());
139         }
140     }
```

```
...
168 service->atribuirCamionistaACamiao(nomeCamionista, matricula);
169 menu.mostrarSucessoAtribuicao(nomeCamionista, matricula);
```

Figura 8 - Excerto de código do GestorController.cpp

### 3.4.3 Singleton

A classe Empresa é implementada como um Singleton porque representa a empresa de transportes, entidade única no sistema. Não faria sentido existir mais do que uma empresa a correr ao mesmo tempo.

```
11 class Empresa {
12 private:
13     static Empresa* instance;
14
15     std::string nif;
16     std::string nome;
17     float coordenadaX;
18     float coordenadaY;
19     CamiaoContainer camiaoContainer;
20     CamionistaContainer camionistaContainer;
21     CargaContainer cargaContainer;
22     RotaContainer rotaContainer;
23     LocalidadeContainer localidadeContainer;
24
25     Empresa(std::string nif, std::string nome);
26
27 public:
28     static Empresa* getInstance(std::string nif = "", std::string nome = "");
```

Figura 9 - Excerto de código do Empresa.h

```
4     Empresa* Empresa::instance = nullptr;
5
6     Empresa::Empresa(std::string nif, std::string nome) {
7         this->nif = nif;
8         this->nome = nome;
9         this->coordenadaX = 0.0f;
10        this->coordenadaY = 0.0f;
11    }
12
13    Empresa* Empresa::getInstance(std::string nif, std::string nome) {
14        if(instance == nullptr){
15            instance = new Empresa(nif, nome);
16        }
17        return instance;
18    }
```

Figura 10 - Excerto de código do Empresa.cpp

No main.cpp, a empresa é inicializada uma única vez com os dados reais:

```
8     Empresa::getInstance("12345678", "Empresa Transportes");
```

Figura 11 - Linha de código do main.cpp



A partir daí, qualquer classe do sistema acede aos containers da empresa através do mesmo `getInstance()`, sem parâmetros, porque a instância já existe:

```
13  GestorService::GestorService(){
14      // vai buscar a instancia da empresa ja criada no main
15      Empresa* empresa = Empresa::getInstance();
16      this->camiaoContainer = &empresa->getCamiaoContainer();
17      this->camionistaContainer = &empresa->getCamionistaContainer();
18      this->cargaContainer = &empresa->getCargaContainer();
19      this->rotaContainer = &empresa->getRotaContainer();
20      this->localidadeContainer = &empresa->getLocalidadeContainer();
21  }
```

Figura 12 - Excerto de código do `GestorService.cpp`

O mesmo padrão repete-se no `CamionistaService` e no `GeneralRepository`, todos acedem à mesma e única `Empresa`, garantindo que todos trabalham sobre os mesmos containers em memória.

### 3.5 MCV (Model-View-Controller)

#### 3.5.1 Model

O Model é composto pelas classes de domínio (`Camiao`, `Camionista`, `Carga`, `Localidade`, `Rota`) e pelos respetivos Containers. São completamente autónomas, não sabem nada sobre a interface nem sobre quem as usa. Guardam dados e expõem métodos que respondem a perguntas de negócio.

```
44  bool Camiao::temCargas(){
45      if(cargas.size() > 0){
46          return true;
47      }
48      else return false;
49  }
50
51  void Camiao::verificarDisponivel(){
52      if(this->getCamionista() != nullptr){
53          throw std::invalid_argument("Camiao ja tem camionista atribuido.");
54      }
55  }
```

Figura 13 - Excerto de código do `Camiao.cpp`

A `Empresa` funciona como raiz do Model via Singleton, sendo o ponto de acesso único a todos os controllers em memória.

### 3.5.2 View

A View é composta pelos três menus (MenuPrincipal, MenuGestor, MenuCamionista). A sua única responsabilidade é apresentar informação e ler input do utilizador. Não toma decisões nem conhece o Model, trabalha exclusivamente com DTO's.

```
24     int mostrarOpcoes();
25     std::string pedirMatricula();
26     bool pedirConfirmacao();
27     void mostrarSucessoRegistarCamiao(std::vector<CamiaoDTO> camioes);
28     void mostrarErro(std::string mensagem);
```

Figura 14 - Excerto de código do MenuGestor.h

### 3.5.3 Controller

O Controller é o orquestrador, recebe eventos da View, delega ao Service e devolve o resultado à View. Não contém lógica de negócio nem lógica de apresentação, apenas lógica de fluxo. O projeto tem três controllers em hierarquia: MenuPrinciaplController despacha para GestorController ou CamionistaController consoante a opção escolhida.

```
68     else if(opcao == 2){
69         std::string nomeCamionista = menu.pedirNomeCamionista();
70         if(nomeCamionista == "v" || nomeCamionista == "V") continue;
71         try{
72             service->registrarCamionista(nomeCamionista);
73             std::vector<CamionistaDTO> camionistas = service->getTodosCamionistas();
74             menu.mostrarSucessoRegistarCamionista(camionistas);
75         }
76         catch(std::invalid_argument &e){
77             menu.mostrarErro(e.what());
78         }
```

Figura 15 - Excerto de código do GestorController.cpp

O fluxo é sempre o mesmo: **View → Controller → Service → Controller → View**.

## 4. Testes

Recorrendo à biblioteca *GoogleTest* é possível verificar se o código responsável por garantir o comportamento pretendido está a funcionar corretamente.

```
7  TEST(CamiaoConstructorTest, ConstrutorValido) {
8      //Arrange
9      bool flag = true;
10     //Act
11     try {
12         Camiao camiao("AB-12-CD", 500.0f);
13     } catch (std::invalid_argument& e) {
14         flag = false;
15     }
16     //Assert
17     EXPECT_TRUE(flag);
18 }
```

Figura 16 - Verificar se ao criar camião (com matrícula e peso válidos) não lança exceção

```
20  TEST(CamiaoValidarMatriculaTest, MatriculaFormatoValido) {
21     //Arrange
22     bool flag = true;
23     //Act
24     try {
25         Camiao::validarMatricula("AB-12-CD");
26     } catch (std::invalid_argument& e) {
27         flag = false;
28     }
29     //Assert
30     EXPECT_TRUE(flag);
31 }
```

Figura 17 - Verificar se o formato "AB-12-CD" é aceite pelo validador

```
5  TEST(CamionistaConstructorTest, ConstrutorValido) {
6      //Arrange
7      bool flag = true;
8      //Act
9      try {
10         Camionista camionista("Joao");
11     } catch (std::invalid_argument& e) {
12         flag = false;
13     }
14     //Assert
15     EXPECT_TRUE(flag);
16 }
```

Figura 18 - Verificar se ao criar camionista (com nome válido) não lança exceção

```

18  TEST(CamionistaConstructorTest, EstadoInicialDisponivel) {
19      //Arrange
20      Camionista camionista("Joao");
21      //Assert
22      EXPECT_EQ(camionista.getEstado(), "Disponivel");
23  }

```

Figura 19 - Verificar se depois de criar camionista é lhe atribuído o estado "Disponível"

```

25  TEST(CamionistaValidarNomeTest, NomeValido) {
26      //Arrange
27      bool flag = true;
28      //Act
29      try {
30          Camionista::validarNome("Joao");
31      } catch (std::invalid_argument& e) {
32          flag = false;
33      }
34      //Assert
35      EXPECT_TRUE(flag);
36  }

```

Figura 20 - Verificar se o validador aceita o nome "Joao"

```

9   TEST(CargaConstructorTest, ConstrutorValido) {
10      // Arrange
11      Localidade loc("Porto", 20.0f, 15.0f);
12      // Act + Assert: criar uma carga com peso valido nao deve lancar excecao
13      EXPECT_NO_THROW(Carga carga(50.0f, &loc));
14  }

```

Figura 21 - Verificar se ao criar uma carga (com peso válido) não lança exceção

```

16  TEST(CargaConstructorTest, EstadoInicialDisponivel) {
17      // Arrange
18      Localidade loc("Porto", 20.0f, 15.0f);
19      Carga carga(50.0f, &loc);
20      // Assert
21      EXPECT_EQ(carga.getEstado(), "Disponivel");
22  }

```

Figura 22 - Verificar se depois de criar carga lhe é atribuída o estado "Disponível"

```

8   TEST(LocalidadeConstructorTest, ConstrutorValido) {
9       // Criar uma localidade normal nao deve lancar excecao
10      EXPECT_NO_THROW(Localidade loc("Porto", 20.0f, 15.0f));
11  }

```

Figura 23 - Verificar se ao criar uma localidade (com nome e coordenadas válidos) não lança exceção

```

42  TEST(LocalidadeCalcularDistancia, DistanciaConhecida) {
43      // Arrange pontos (0,0) e (3,4) -> distancia esperada = 5 (triangulo 3-4-5)
44      Localidade a("A", 0.0f, 0.0f);
45      Localidade b("B", 3.0f, 4.0f);
46      // Act
47      float dist = a.calcularDistancia(b);
48      // Assert
49      EXPECT_NEAR(dist, 5.0f, 0.001f);
50  }

```

Figura 24 - Verificar se a distância é calculada corretamente entre duas localidades

```

5   TEST(RotaConstructorTest, ConstrutorValido) {
6       //Arrange
7       std::vector<std::string> destinos = {"Lisboa", "Porto"};
8       bool flag = true;
9       //Act
10      try {
11          Rota rota(1, "Joao", "AB-12-CD", 150.0f, destinos);
12      } catch (std::invalid_argument& e) {
13          flag = false;
14      }
15      //Assert
16      EXPECT_TRUE(flag);
17  }

```

Figura 25 - Verificar se ao criar uma rota (com os dados completos) não lança exceção

```

19  TEST(RotaConstructorTest, AtributosGuardadosCorretamente) {
20      //Arrange
21      std::vector<std::string> destinos = {"Lisboa"};
22      Rota rota(1, "Joao", "AB-12-CD", 150.0f, destinos);
23      //Assert
24      EXPECT_EQ(rota.getIdRota(), 1);
25      EXPECT_EQ(rota.getNomeCamionista(), "Joao");
26      EXPECT_EQ(rota.getMatriculaCamiao(), "AB-12-CD");
27      EXPECT_FLOAT_EQ(rota.getDistanciaTotal(), 150.0f);
28  }

```

Figura 26 - Verificar se os atributos (ID, nome do camionista, matrícula e distância total) são guardados e acessíveis

#### 4.1 Tabela de taxa de sucesso dos testes

| Teste                    | Percentagem | Nota  |
|--------------------------|-------------|-------|
| Construtor Camiao        | 100         | ----- |
| Formato Matricula        | 100         | ----- |
| Construtor Camionista    | 100         | ----- |
| Camonista Estado Inicial | 100         | ----- |
| Nome Camionista Valido   | 100         | ----- |
| Construtor Carga         | 100         | ----- |
| Carga Estado Inicial     | 100         | ----- |
| Construtor Localidade    | 100         | ----- |
| Cálculo Distancia        | 100         | ----- |
| Construtor Rota          | 100         | ----- |
| Atributos Rota Guardados | 100         | ----- |

## 5. Conclusão

A terceira iteração do projeto de desenvolvimento de software em C++ permitiu consolidar com sucesso o design e a implementação de uma aplicação robusta para a gestão de frotas e entregas logísticas. O cumprimento rigoroso dos requisitos propostos assegurou o controlo dinâmico de camiões, camionistas e cargas, operacionalizando com eficácia regras de negócio essenciais, tais como a atribuição automática de estados de disponibilidade e o planeamento de rotas baseado numa abordagem FIFO.

A nível de arquitetura, o sistema evidenciou elevada maturidade através da separação clara de responsabilidades promovida pelo padrão Model-View-Controller (MVC) e por princípios GRASP cruciais, nomeadamente o *Creator*, *Controller* e *Singleton*. Adicionalmente, o uso combinado de DTO's e Mappers garantiu o correto encerramento das entidades de domínio, enquanto a autonomização da persistência de dados em ficheiros de texto, através do padrão *Repository*, deixou a aplicação perfeitamente separada e flexível para futuras migrações.

Por fim, a fiabilidade do software foi rigorosamente comprovada pela implementação de testes automatizados com a biblioteca GoogleTest. A obtenção de uma taxa de sucesso de 100% em todos os cenários testados, incluindo a validação de formatos e o tratamento de exceções, atesta a qualidade técnica e a estabilidade do software desenvolvido.